

UNIT-2

Knowledge Representation -

A field in AI that studies how to encode knowledge about the world into a structured form so that a computer can reason with it, make decisions, and answer questions

- (i) Choosing the right level of abstraction
 - (ii) Declarative vs procedural knowledge
 - (iii) Incomplete / uncertain knowledge
 - (iv) Consistency and computational efficiency
- } Issues

KR Issues:

1) Choosing the right level of abstraction:

Selecting how detailed or simplified the knowledge should be so that it is easy to represent and reason with

- (a) Too much detail $\rightarrow$ complex and slow reasoning
- (b) Too little detail $\rightarrow$ incomplete or incorrect conclusions

2) Declarative vs procedural knowledge:

(a) Declarative: "Know-what" - facts and information about the world

eg - A dog is an animal

(b) Procedural: "Know-how" - steps or procedures to perform tasks

eg - How to solve a puzzle

3) Incomplete / uncertain knowledge:

Knowledge may be missing, uncertain, or noisy.

4) Consistency and computational Efficiency:

(a) Consistency: No contradictions should exist in the knowledge base

(b) Computational efficiency: Reasoning should be fast enough to be practical

Predicate Logic (First Order):

(i) Represents knowledge (logic) using predicates, variables, quantifiers ($\forall, \exists$).

(ii) More expressive than propositional logic

(iii) Terminologies:

Propositional logic vs Predicate logic

Syntax & Semantics

Quantifiers ($\forall, \exists$)

Representing facts & rules

- eg- (i) $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$
(ii) $\text{Human}(\text{Socrates}) \rightarrow \text{Infer Mortal}(\text{Socrates})$

Logic Programming:

- (i) Uses logic to write programs (eg, Prolog)
- (ii) Programs are sets of facts and rules, and execution is ~~important~~ inference

eg- Prolog:

father(john, mary)

parent(X, Y) :- father(X, Y)

Query: parent(john, mary) ? $\rightarrow$ true

(iii) Logic as a programming language

(iv) Prolog: facts, rules, queries

(v) Horn clauses

(vi) Backward chaining

(programs are written as logical statements & system uses inference to determine solutions)

Prolog (most widely used logic)

(a) Facts = basic true statements about the world

eg: parent(john, mary)

(b) Rules = conditional statements defining relationships

eg: grandparent(X, Y) :- parent(X, Z), parent(Z, Y)

(c) Queries = questions asked to the system

eg: ?- grandparent(john, Y)

Prolog answers queries by logically deriving conclusions from facts and rules

Horn clauses:

Special form of logical statements used in logic programming.

They have at most 1 positive (true) literal, making them easy for computers to process.

eg- $a :- b, c$ ("a is true if b and c are true.")

Horn clauses enable efficient reasoning in Prolog

Backward chaining:

A reasoning method that starts with a goal & works backward to find supporting facts.

Used by Prolog for answering queries

It tries to prove rules & find matching facts step by step.

Semantic Nets -

- A knowledge representation technique that organizes information in a graphical way to show relationships between concepts.

- Graph based representation

- Nodes = concepts

- Edges = relationships

- eg - Canary $\rightarrow$ is-a $\rightarrow$ Bird

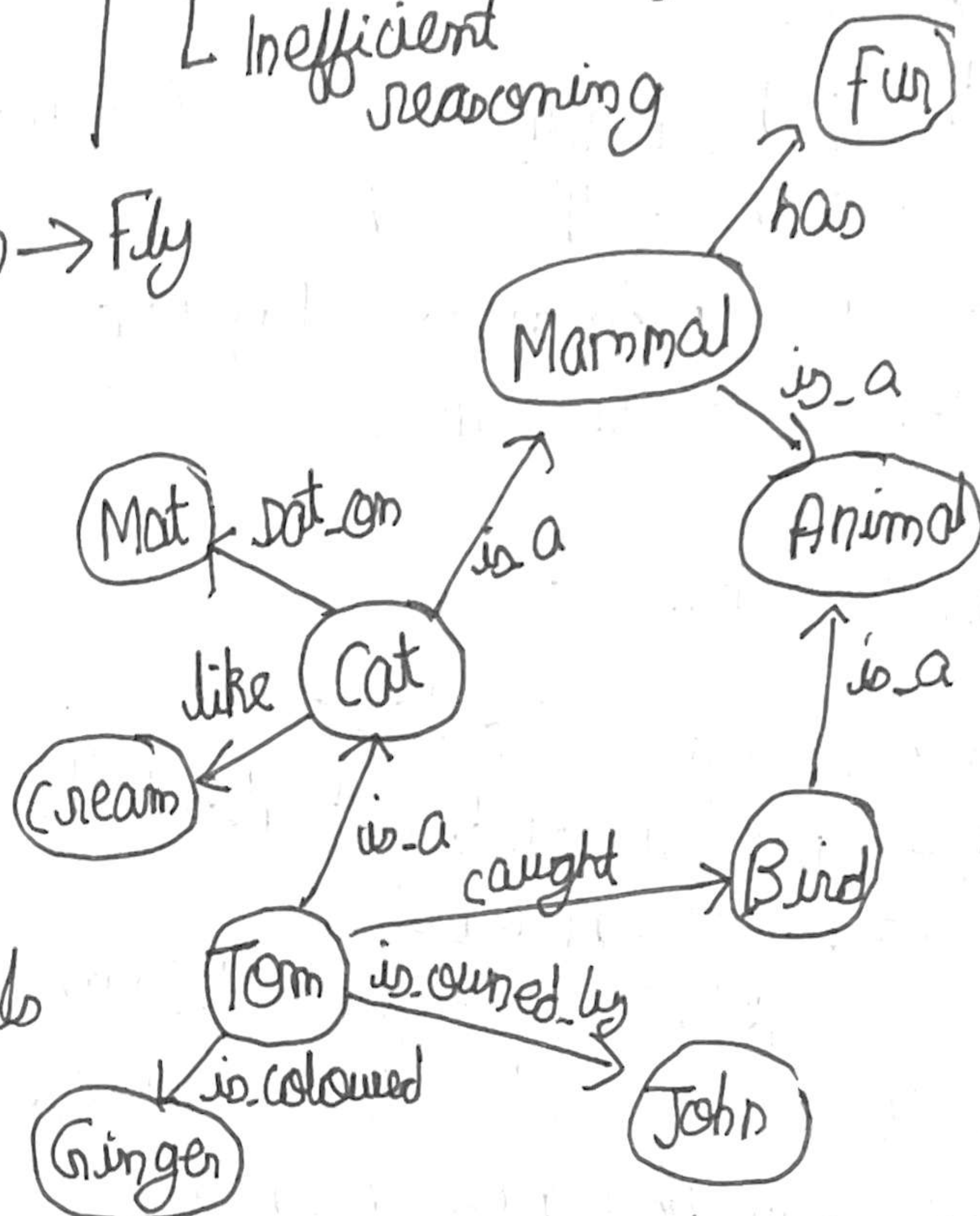
Bird $\rightarrow$ Can $\rightarrow$ Fly

Inference: Canary $\rightarrow$ Can $\rightarrow$ Fly

- Limitations:

- Ambiguity in representation
- Scalability issues
- Inheritance conflicts
- Inefficient reasoning

eg. Tom is a cat
Tom caught a bird
Tom is owned by John
Cats like cream
Cat sat on the mat
Cat is a mammal
Bird is an animal
All mammals are animals
Mammals have fur



Frames -

- Data structures for representing stereotyped situations (like OOP objects)

- Contains slots (attributes) and values

Inheritance:

- Subclasses inherit properties from super classes

eg. Frame: Dog

IS-A: Animal

Legs: 4

Sound: Bark

Poodle inherits Legs: 4 and Sound: Bark from Dog

Frame = structured objects with slots

Default values

~~strict~~ Types of inheritance (strict, multiple)

Applications in expert systems

Aspect	Semantic Networks	Frames
1) Representation	Graphical representation with nodes and edges	Data structures with slots and values
2) Components	Nodes (concepts) Edges (relationships)	Slots (attributes) Values (fillers)
3) Structure	Hierarchical or associative network	Structured, focused on specific entities
4) Purpose	Represents relationships between concepts	Models stereotypical knowledge of objects
5) Flexibility	More flexible	More rigid
6) Reasoning capability	Inference through relationships	Procedural attachments for reasoning
7) Modification	More complex due to interconnected nodes	Easier to modify by adjusting slots & values

Constraint Propagation -

- Solves problems by propagating constraints across variables.
 - Constraints limit possible variable values
 - Used in CSP problems
 - Forward checking and arc consistency
- eg - Sudoku: If a no is ~~eliminated~~ placed in a cell, it's eliminated from all other related cells.

Reasoning Programming -

- Refers to writing programs that are capable of drawing logical conclusions, solving problems, or making decisions by using rules, facts & inference mechanisms.

- Components:

(a) Logical Reasoning:

- Programs are based on formal logic (eg - propositional logic, first order logic)
- They can infer new knowledge from existing facts & rules

(b) Rule based programming:

- Logic is encoded as rules:

if condition, then action!

eg - If $\text{Human}(x) \rightarrow \text{Mortal}(x)$ Conclusion
 $\text{Human}(\text{Socrates})$
 $\Rightarrow \text{Mortal}(\text{Socrates})$

(c) Inference Engine:
A component that applies rules to known facts to derive new facts

Techniques: forward chaining, Backward chaining, Resolution

(d) Declarative Programming:

Focuses on what to solve, not how to solve it

Eg- Prolog, Datalog, Answer Set Programming (ASP)

#Belief (B) -

- (i) Degree of confidence in a statement or hypothesis
- (ii) Based on evidence, reasoning, or experience
- (iii) ~~Data not at~~ Does not always require complete proof
- (iv) Represents justified confidence in a proposition
- (v) In evidence theory:

~~Bel~~ Bel(H) = how strongly evidence supports hypothesis H

#Hypothesis -

- (i) A proposed explanation or assumption
- (ii) Based on limited evidence
- (iii) Can be tested through experiments or observations
- (iv) In probability: a possible state tested against evidence

Eg- (a) H1: Intrusion exists
(b) H2: Traffic is normal

#Likelihood -

- (i) Measures how strongly evidence supports a hypothesis
- (ii) Not the probability of hypothesis
- (iii) Shows how probable the evidence is given the hypothesis

Summary:

Term	Meaning
1) Belief	Confidence in hypothesis
2) Hypothesis	Testable assumption
3) Likelihood	Support provided by evidence

- ## # Reasoning under uncertainty -
- 1) Need for uncertainty handling
 - 2) Sources of uncertainty
 - 3) Deterministic vs probabilistic reasoning

Reasoning under uncertainty -

- Used when information is incomplete or uncertain
- Helps make decisions using evidence & probability

- 1) Probability Theory:
 - (i) Assigns probabilities to uncertain events
 - (ii) Provides mathematical foundation for uncertainty
 - (iii) Based on rules of probability
- 2) Bayesian Inference:
 - (i) Based on Bayes Theorem
 - (ii) Updates beliefs using new evidence
 - (iii) Used in diagnosis, ML, expert systems
- 3) Dempster-Shafer Theory:
 - (i) Alternative to probability theory
 - (ii) Uses belief and plausibility values
 - (iii) Combines evidence from multiple sources
 - (iv) Handles partial & conflicting information

Probability Review -

- 1) Conditional Probability:
 - (i) Probability of an event A occurring given that another event B has already occurred
 - (ii) Notation: $P(A|B)$
eg - If 30 out of 100 students passed Math, and 20 of those also passed Science:
$$P(\text{Science}|\text{Math}) = 20/30 = 0.67$$
- 2) Joint Probability:
 - (i) The probability that 2 events occur together
 - (ii) Notation: $P(A \cap B)$
eg - If $P(M) = 0.3$ and $P(S) = 0.2$ then
$$P(M \text{ and } S) = 0.2$$

3) Independence:

(i) 2 events A and B are independent if the occurrence of one does not affect the probability of the other

(ii) Condition: $P(A \cap B) = P(A) \cdot P(B)$

Eg - If $P(A) = 0.5$ (coin landing heads)

$P(B) = 0.5$ (another INDEPENDENT coin landing heads)

then: $P(A \cap B) = 0.5 \times 0.5 = 0.25$

4) Bayes Theorem:

(i) A formula to update probability of an event using new evidence

(ii) Formula:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Eg - Medical Test:

Probability of disease: $P(D) = 0.01$

Test accuracy if person has disease: $P(T|D) = 0.9$

Probability of positive test overall: $P(T) = 0.05$

Then: $P(D|T) = \frac{0.9 \times 0.01}{0.05} = 0.18$

So even after a positive test, chances of having the disease is only 18%.

(iii) Used in:

- Medical diagnosis
- Spam filtering
- Classification
- ML models

5) Dempster Shafer Theory:

(i) A generalization of probability theory for reasoning with uncertain, incomplete, or ambiguous information

(ii) Key concepts:

- Belief (Bel)

- ~~Plausibility~~ Plausibility (Pl): Max possible support (not contradicted)

- Mass Function (m): Assigns probability to sets of outcomes, not just singletons

(iii) Advantages:

- └ Allows representing uncertainty without requiring exact probabilities
- └ Useful when evidence is weak or comes from multiple unreliable sources

First Order Predicate Logic (FOPL) —

1) Introduction:

- (i) First Order Logic (FOL), also called Predicate Logic
- (ii) Used in AI to represent knowledge in a structured logical form
- (iii) A predicate represents a property or relationship between objects

2) Predicates:

- (i) Describes a property of an object or a relation between objects

(ii) eg:

Human(x) $\rightarrow$ x is human

Loves(x, y) $\rightarrow$ x loves y

3) Arity:

- (i) Unary $\rightarrow$ one argument (Tall(x))
- (ii) Binary $\rightarrow$ two arguments (Friend(x, y))
- (iii) n-ary $\rightarrow$ multiple arguments (Between(x, y, z))

4) Predicates vs Propositional Logic:

- (i) Propositional Logic: simple true/false statements (P, Q)
- (ii) First Order Logic: uses predicates with variables to describe relationships between objects

5) ~~eg~~ Example of FOL statement:

(i) Domain: People

(ii) Predicate: Parent(x, y)

eg statement: $\forall x \forall y (\text{Parent}(x, y) \rightarrow \text{Human}(x))$

Meaning: If x is parent of y , then x is human

6) Key components:

(a) Constants:

- └ Represents specific objects in a domain
- └ eg - John, Paris, 3

(b) Variables:

- └ Represents general or unknown objects
- └ eg - x, y, z

(c) Predicates:

- └ Represents properties or relationships
- └ eg - Likes (John, Pizza)

(d) Functions:

- └ Maps objects to another object
- └ eg - MotherOf (x)

(e) Quantifiers:

- └ Universal ($\forall$) $\rightarrow$ for all
- └ Existential ($\exists$) $\rightarrow$ there exists

(f) Logical connectives:

- └ $\wedge$ (AND)
- └ $\vee$ (OR)
- └ $\neg$ (NOT)
- └ $\rightarrow$ (IMPLIES)
- └ $\Leftrightarrow$ (IFF)

7) Syntax of FOL:

(i) Atomic formula: basic predicate statement

eg. Father (John, x)

(ii) Complex formula: built using quantifiers & logical connectives

eg. $\forall x, (\text{Student}(x) \rightarrow \text{Studies}(x, \text{Subject}))$

8) Semantics of FOL:

Defines the meaning of formulas

- └ Domain (D): set of objects in discussion

- └ Interpretation (I): assigns meaning to constants, functions, & predicates.

- └ Formula is true if it correctly represents relations in the domain

9) Use of FOL in AI:

(i) Knowledge representation:

- └ Represents facts about the ~~world~~ world logically

- └ eg. $\forall x (\text{Bird}(x) \rightarrow \text{CanFly}(x))$

(ii) Automated Reasoning:

- └ Uses ~~inference~~ inference techniques like resolution & unification to derive new knowledge

(iii) Expert Systems:

- └ Encodes domain knowledge using logical rules

(iv) Natural Language Understanding:

- └ Converts natural language sentences into logical representations

Inference in FOL -

- (i) Process of deriving new conclusions from known facts & rules
- (ii) Enabled systems to reason, answer queries, & deduce knowledge

Main Inference Techniques in FOL -

1) Modus Ponens:

- (i) Basic rule of inference
- (ii) Rule: $P \rightarrow Q, P \Rightarrow Q$
- (iii) eg -
 - If it rains $\rightarrow$ ground is wet
 - It rains
 - Therefore $\rightarrow$ ground is wet

3) Resolution:

- (i) Rule of inference used in automated theorems proving
- (ii) Works by refutation
- (iii) Steps:
 - Assumes negation of statement to be proved
 - Adds it to the knowledge base
 - Apply resolution repeatedly
 - If a contradiction appears $\rightarrow$ statement is true

2) Unification:

- (i) Process of making 2 logical expressions identical by finding substitutions
- (ii) Used in automated reasoning & logic programming
- (iii) eg - Knows(John, x)
Knows(John, Mary)
Substitution: $x = \text{Mary}$
Unifier: $\{x/\text{Mary}\}$

Types of Inference in FOL -

1) Deductive Inference:

- (i) Derives logically certain conclusions from known facts

(ii) eg -

Known:

$\forall x (\text{Bird}(x) \rightarrow \text{CanFly}(x)), \text{Bird}(\text{Tweety})$

Conclusion:

$\text{CanFly}(\text{Tweety})$

2) Universal Instantiation:

- (i) Replaces a universally quantified variable with a specific constant

(ii) eg -

$\forall x (\text{Student}(x) \rightarrow \text{Studies}(x))$

Infer:

$\text{Student}(\text{John}) \rightarrow \text{Studies}(\text{John})$

3) Existential Instantiation:

- (i) Replaces an existential variable with a new constant

(ii) eg -

$\exists x \text{Animal}(x)$

Infer:

$\text{Animal}(a)$

(iv) eg Rule:

Clauses:

$A \vee B$

$\neg B \vee C$

Inference: $A \vee C$

(eliminates complementary variables)

Conversion to Predicate Logic -

Constants: Ram, 5, India

Variables: x, y, z

Predicates: Student(x), Likes(x, y)

Quantifiers: $\forall$ (for all), $\exists$ (there exists)

Logical Symbols:

$\neg \rightarrow$ NOT

$\wedge \rightarrow$ AND

$\vee \rightarrow$ OR

$\rightarrow \rightarrow$ IMPLIES

$\leftrightarrow \rightarrow$ IF AND ONLY IF

1) Statement:

English = Ram is a student

Predicate = Student(x)

Logical Form = Student(Ram)

2) Universal Quantifier:

English = All students study

Predicates = Student(x), Studies(x)

Logical form = $\forall x (\text{Student}(x) \rightarrow \text{Studies}(x))$

3) Existential Quantifier:

English = Some students are intelligent

Predicates = Student(x), Intelligent(x)

Logical Form = $\exists x (\text{Student}(x) \wedge \text{Intelligent}(x))$

4) Relationship:

English = No student is lazy

Predicate = Mother(y, x)

Logical Form = $\forall x \exists y \text{Mother}(y, x)$

5) Negation:

English = No student is lazy

Logical Form = $\forall x (\text{Student}(x) \rightarrow \neg \text{Lazy}(x))$

Equivalent Form = $\neg \exists x (\text{Student}(x) \wedge \text{Lazy}(x))$

Steps to Convert English to Predicate Logic

Q. "All students study"

1) Define Domain (Domain: All people)

2) Define Predicates

└ Student(x) $\rightarrow$ x is a student

└ Studies(x) $\rightarrow$ x studies

3) Identify Quantifiers

└ All $\rightarrow$ Universal ($\forall$)

4) Write logical expression for

└ $\forall x (\text{Student}(x) \rightarrow \text{Studies}(x))$

5) Verify meaning

Belief vs Hypothesis

Feature	Belief
1) Nature	Degree of confidence
2) Based on	Probability & evidence
3) Used in	Probabilistic reasoning
4) Dynamic	Continuously updated
5) Eg.	80% chance of rain

Hypothesis
Proposed model/explanation
Training data
Machine Learning
Selected after evaluation
Linear regression model

Resolution-Based Inference Process - Steps

- (i) Convert all statements into clausal form
- (ii) Apply resolution rule repeatedly
- (iii) Derive new clauses
- (iv) If empty clause appears $\Rightarrow$ proof successful

Example of Deductive Reasoning in FOL -

- (i) Knowledge Base:
 - $\vdash \forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$
 - $\vdash \text{Human}(\text{Socrates})$
- (ii) Goal:
 - $\text{Mortal}(\text{Socrates})$
- (iii) Steps:
 - $\vdash$ Instantiate universal rule with $x = \text{Socrates}$
 - $\vdash \text{Human}(\text{Socrates}) \rightarrow \text{Mortal}(\text{Socrates})$
 - $\vdash$ Apply Modus Ponens
- (iv) Conclusion:
 - $\text{Mortal}(\text{Socrates})$

Importance -

- (i) Used in expert systems, automated reasoning, planning & NLP
- (ii) Helps simulate human like thinking

Example FOL Statements -

English Sentence	FOL Expression
1) All humans are mortal	$\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$
2) Socrates is a human	$\text{Human}(\text{Socrates})$
3) Therefore, Socrates is mortal	$\text{Mortal}(\text{Socrates})$
4) There is a cat on the roof	$\exists x (\text{Cat}(x) \wedge \text{OnRoof}(x))$

Importance Example of Inference -

- 1) Given: $\forall x (\text{Man}(x) \rightarrow \text{Mortal}(x))$ $\text{Man}(\text{Socrates})$
- 2) Apply Universal Instantiation to (1):
 - $\text{Man}(\text{Socrates}) \rightarrow \text{Mortal}(\text{Socrates})$
- 3) Apply Modus Ponens with (2):
 - $\Rightarrow \text{Mortal}(\text{Socrates})$

Propositional Inference vs First Order Inference (Predicate) -

Feature	Propositional Inference	First Order Inference
1) Logic Type	Propositional (Boolean) Logic	Predicate (First Order) Logic
2) Knowledge Expressiveness	Limited	Highly expressive
3) Efficiency	Faster but less powerful	More powerful but computationally complex
4) Representation	Uses simple symbols like P, Q, R for whole statements	Uses predicates, variables, functions & quantifiers
5) Quantifiers	Not supported	Supports $\forall$ (for all) and $\exists$ (there exists)
6) Used for	Simple rule based systems	Real world reasoning, expert systems, theorem solving
7) Granularity	Coarse = treats entire statements as single units	Fine = represents internal structure of a statement

Unification in FOL -

- (i) Process of finding a substitution for variables that makes 2 predicates identical
- (ii) Purpose:
- Matches facts and rules during inference.
 - Used in forward chaining, backward chaining & resolution

(iii) eg:

Given:

- Rule: Likes(x, IceCream)

- Fact: Likes(John, y)

Substitution: $\{x = \text{John}, y = \text{IceCream}\}$

Result: Likes(John, IceCream)

Both predicates become identical

(iv) When Unification

Fails:

- Occurs when constants do not match

eg:

- Likes(John, Pizza)

- Likes(Mary, Pizza)

Since John $\neq$ Mary,
Unification fails

Q. Inference using Unification:

And Rule: $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$

Fact: $\text{Human}(\text{Socrates})$

Step 1: Unification

Match: $\text{Human}(x)$ with $\text{Human}(\text{Socrates})$

Substitution: $\{x = \text{Socrates}\}$

Step 2: Apply Rule

Using substitution in the rule:

$\text{Human}(\text{Socrates}) \rightarrow \text{Mortal}(\text{Socrates})$

Step 3: Infer Conclusion

Since $\text{Human}(\text{Socrates})$ is true, infer:

$\text{Mortal}(\text{Socrates})$

Step 4: Update Knowledge Base

Add the new fact:

$\text{Mortal}(\text{Socrates})$

Backward Chaining -

(i) Goal driven inference method used in FOL and rule based systems

(ii) Starts with a goal (query) & works backward using rules & facts to prove it

(iii) Steps:

- Start with the goal to be proved

- Find rules that conclude the goal

- Treat rule premises as sub goals

- Try to prove each sub goal using known facts or rules

- If all sub goals are satisfied, the goal is proven

(iv) Example:

Knowledge Base:

Rule: $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$

Fact: $\text{Human}(\text{Socrates})$

Goal: $\text{Mortal}(\text{Socrates})$

Inference:

- Goal $\rightarrow \text{Mortal}(\text{Socrates})$

- Rule found $\rightarrow \text{Human}(x) \rightarrow \text{Mortal}(x)$

- Sub-goal $\rightarrow \text{Human}(\text{Socrates})$

- Fact exists $\rightarrow \text{Human}(\text{Socrates})$

- Therefore $\rightarrow \text{Mortal}(\text{Socrates})$

(v) Advantages:

- Efficient for answering specific queries

- Searches only paths relevant to the goal

(vi) Disadvantages:

- Can lead to infinite loops if recursion is not controlled

- Requires complex matching & rule searching

(vii) Applications:

- Expert systems

- AI reasoning engines (Prolog)

- Logic Programming

Lifted Forward Chaining -

(i) A rule based inference method in FOL that uses variables and unification to derive new facts

(ii) Steps

- Match rule premises with known facts using unification
- Instantiate the rule with substitutions
- Add the new inferred fact to the knowledge base
- Repeat until no new facts can be derived

(iii)

Resolution -

- Inference rule used in automated reasoning

- Core idea

- Based on proof by contradiction
- To prove a statement:
 - Negate the rule
 - Try to derive a contradiction

If contradiction occurs $\Rightarrow$ original statement is True

- Important concepts

(i) Clause

Disjunction (OR) of literals

eg- $A \vee B$

$\neg A \vee C$

(ii) Literal

Either: A or $\neg A$

(iii) Resolution Rule

If 2 clauses contain complementary literals

$A \vee B$

$\neg A \vee C$

Resolve on A

$B \vee C$

We remove A and $\neg A$

- Advantages:

- Mechanically applicable
- Works well with automated deduction

- Disadvantages:

- Requires formulas in CNF
- Can be computationally expensive
- May derive many irrelevant clauses in complex systems

Backward Chaining Example

Knowledge Base

Rule: $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$

Fact: $\text{Human}(\text{Socrates})$

Goal: Prove: $\text{Mortal}(\text{Socrates})$

Steps:

- 1) Start with goal: $\text{Mortal}(\text{Socrates})$
- 2) Find rule that concludes it: $\text{Human}(x) \rightarrow \text{Mortal}(x)$
- 3) Sub goal becomes: $\text{Human}(\text{Socrates})$
- 4) Check facts: $\text{Human}(\text{Socrates})$ ~~becomes~~ is true
- 5) Therefore: $\text{Mortal}(\text{Socrates})$ is proven

What is CNF (Conjunctive Normal Form)

Each clause contains ORs only

$$(Clause 1) \wedge (Clause 2) \wedge (Clause 3)$$

$$eg - (A \vee B) \wedge (\neg A \vee C)$$

Q. How to convert to CNF?

Ans 1) Remove implication

$$A \rightarrow B = \neg A \vee B$$

2) Move negations inward (De Morgan rules)

$$\neg(A \wedge B) = \neg A \vee \neg B$$

$$\neg(A \vee B) = \neg A \wedge \neg B$$

3) Standardize variables

4) Convert to AND of OR clauses

$$(Clause 1) \wedge (Clause 2) \wedge (Clause 3)$$

Each clause must contain OR only

Resolution algorithm steps

- 1) Convert all statements to First Order Logic
- 2) Negate the goal
- 3) Convert all formulas to CNF
- 4) Standardize variables
- 5) Apply unification
- 6) Apply resolution rules repeatedly
- 7) If empty clause ($\perp$) appears $\rightarrow$ goal proven

Skolemization (FOL) -

(i) Definition

- Process of eliminating existential quantifiers
- Replace with skolem constants or skolem functions
- Used during conversion to CNF for resolution

(ii) Purpose

- Required for resolution & automated theorem proving
- Converts formulas to CNF with only $\forall$
- Simplifies reasoning by removing existential variables

(iii) Rules

- Rule 1: $\exists$ not inside $\forall$ (Replace with Skolem constant)

$$eg: \exists x P(x) \rightarrow P(c)$$

- Rule 2: $\exists$ inside $\forall$ (Replace with Skolem function)

$$eg: \forall y \exists x P(x, y) \rightarrow \forall y P(f(y), y)$$

(iv) Example

Original

$$\forall y \exists x \text{Loves}(x, y)$$

After Skolemization

$$\forall y \text{Loves}(f(y), y)$$

Example of Resolution -

(IOM) (SM)

1) Knowledge base

$\vdash \text{Human}(\text{Socrates})$

$\vdash \forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$

~~Goal: Prove Mortal(Socrates)~~

Goal: Prove Mortal(Socrates)

Steps:

(i) Negate goal: $\neg \text{Mortal}(\text{Socrates})$

(ii) Convert to CNF:

$\vdash \text{Human}(\text{Socrates})$

$\vdash \neg \text{Human}(x) \vee \text{Mortal}(x)$

$\vdash \neg \text{Mortal}(\text{Socrates})$ (negated goal)

(iii) Apply resolution:

$\vdash$ Unify $x = \text{Socrates}$ in rule 2: $\neg \text{Human}(\text{Socrates}) \vee \text{Mortal}(\text{Socrates})$

$\vdash$ Resolve with $\text{Human}(\text{Socrates}) \rightarrow \text{Mortal}(\text{Socrates})$ (Socrates)

$\vdash$ Now resolve with $\neg \text{Mortal}(\text{Socrates}) \rightarrow$ Empty clause ($\perp$)

(iv) Hence, Mortal(Socrates) is proven

2) Knowledge base (English)

$\vdash$ Every human is mortal

$\vdash$ Anyone who kills a human is a criminal

$\vdash$ John kills Socrates

$\vdash$ Socrates is a human

Goal: John is a criminal

Steps:

(i) Convert statements into predicate logic

Let, $\text{Human}(x) \rightarrow x$ is Human

$\text{Mortal}(x) \rightarrow x$ is mortal

$\text{Kills}(x, y) \rightarrow x$ kills y

$\text{Criminal}(x) \rightarrow x$ is criminal

Logical representation:

$\vdash \forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$

$\vdash \forall x \forall y ((\text{Kills}(x, y) \wedge \text{Human}(y)) \rightarrow \text{Criminal}(x))$

$\vdash \text{Kills}(\text{John}, \text{Socrates})$

$\vdash \text{Human}(\text{Socrates})$

Negation of conclusion

$\vdash \neg \text{Criminal}(\text{John})$

(ii) Remove implications

Use rule: $P \rightarrow Q \equiv \neg P \vee Q$

$$(a) \neg \text{Human}(x) \vee \text{Mortal}(x)$$

$$(b) \neg (\text{Kills}(x, y) \wedge \text{Human}(y)) \vee \text{Criminal}(x)$$

Apply De Morgan's law:

$$\neg \text{Kills}(x, y) \vee \neg \text{Human}(y) \vee \text{Criminal}(x)$$

(iii) Convert to Clause Form (CNF)

Clauses

$$C1: \neg \text{Human}(x) \vee \text{Mortal}(x)$$

$$C2: \neg \text{Kills}(x, y) \vee \neg \text{Human}(y) \vee \text{Criminal}(x)$$

$$C3: \text{Kills}(\text{John}, \text{Socrates})$$

$$C4: \text{Human}(\text{Socrates})$$

$$C5: \neg \text{Criminal}(\text{John})$$

(iv) Skolemization

There are no existential quantifiers here, so Skolemization is not required in this example.

(v) Resolution

(a) Resolve C2 and C3

$$C2: \neg \text{Kills}(x, y) \vee \neg \text{Human}(y) \vee \text{Criminal}(x)$$

$$C3: \text{Kills}(\text{John}, \text{Socrates})$$

$$\text{Substitute: } x = \text{John} \\ y = \text{Socrates}$$

Result:

$$C6: \neg \text{Human}(\text{Socrates}) \vee \text{Criminal}(\text{John})$$

(b) Resolve C6 and C4

$$C6: \neg \text{Human}(\text{Socrates}) \vee \text{Criminal}(\text{John})$$

$$C4: \text{Human}(\text{Socrates})$$

Resolving gives:

$$C7: \text{Criminal}(\text{John})$$

(c) Resolve C7 and C5

$$C7: \text{Criminal}(\text{John})$$

$$C5: \neg \text{Criminal}(\text{John})$$

$$\text{Result: } \square (\text{Empty Clause})$$

(vi) Conclusion

Since the empty clause ($\square$) is derived, the negated of the conclusion is false.

Therefore: John is a criminal.

Advantages of FOL

- (i) Expressive power
- (ii) Scalability
- (iii) Reusability

Limitations of FOL

- (i) Computational complexity
- (ii) Incompleteness
- (iii) Requires domain knowledge

Rule Based Systems and Learning -

1) Representing Knowledge using rules

- Knowledge stored as IF-THEN rules
- Rules consists of antecedent (condition) & consequent (action/outcome)
- Eg- IF temperature > 100 THEN fan = ON

2) Forward Chaining

(i) Definition

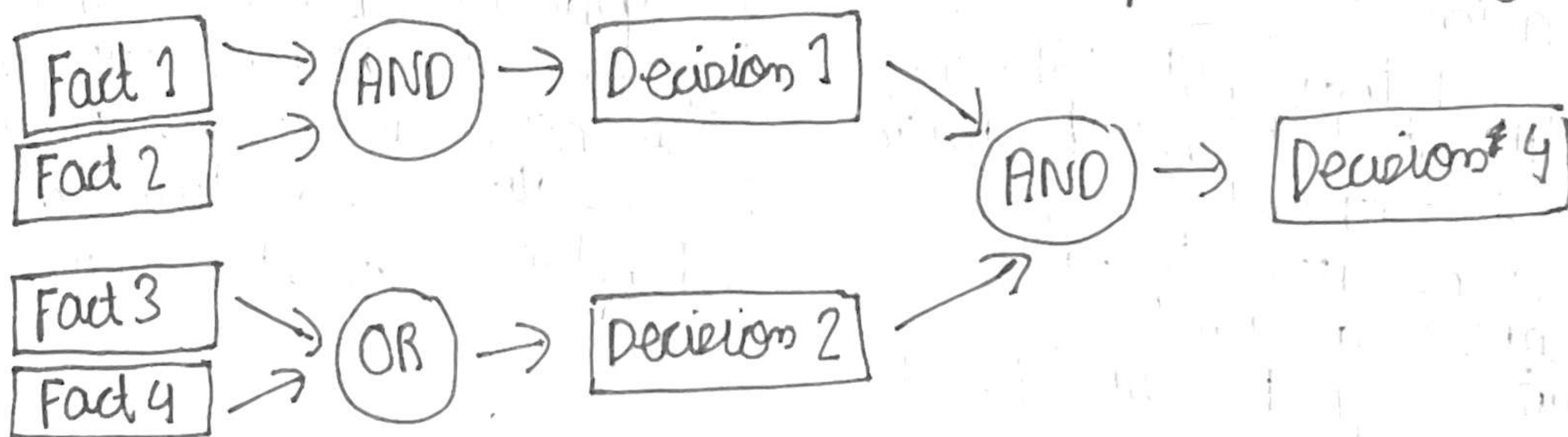
- Data driven reasoning
- Starts with known facts & applies rules to infer new facts
- Stops when goal is reached or no new facts can be derived

(ii) Process Steps

- More initial facts in Working Memory (WM)
- Match facts with IF-THEN rules in the rule base
- Select rules whose conditions are satisfied
- Fire rule → adds new facts to Working Memory
- Repeat Match → Fire → Add facts until no new facts (saturation)

(iii) Applications

- Expert systems
- Monitoring & Control systems
- Fault diagnosis
- Intrusion detection
- Automation systems



3) Backward Chaining

(i) Definition

- Goal driven inference technique
- Starts with a goal/query to prove
- Searches rule backward
- Breaks goal into sub goals
- Stops when sub goals are satisfied or no rule/fact exists

(ii) Eg- Rule base

- R1: fever & cough → flu
- R2: flu → advise-rest
- R3: rash → measles

goal: advise-rest

(iii) Steps

- Goal: advise-rest
- Use rule R2 → sub-goal flu
- Use rule R1 → sub goals fever, cough
- Facts: fever = true, cough = true
- Sub goals satisfied → Goal proven
- Result: advise-rest = true

4) Rule Based Deductive Systems

- (i) ~~Works~~ System derive new knowledge using Rule base + Inference Engine
- (ii) Working memory stores current facts
- (iii) Inference engine steps:
 - Matching: finds rules matching facts
 - Selection: choose rule to apply
 - Execution: apply rule and update ~~or~~ facts
- (iv) Process repeats iteratively until goal reached or no rule applies
- (v) Used in expert systems, diagnostic systems, automated reasoning

5) Lifted Forward Chaining

- (i) Forward chaining using variables & generalized rules
- (ii) Works with FOL level
- (iii) Uses unification to match rules with facts
- (iv) Avoids generating all possible instances $\rightarrow$ more efficient inference
- (v) Used in knowledge bases, ontologies, AI planning

Inductive Learning -

- (i) Learn general rules from specific examples
- (ii) Core idea: ~~specific~~ specific $\rightarrow$ general
- (iii) Forms hypothesis/models that generalize to unseen data
- (iv) Used in classification, pattern recognition, concept learning
- (v) Searches hypothesis space to find best rule/model
- (vi) Performance depends on quality & quantity of training data

Decision Trees -

- (i) Model that splits data using attribute tests to make decisions
- (ii) Nodes: attribute
- (iii) Branches: attribute value
- (iv) Leaves: class labels
- (v) Built using metrics like Info Gain & Gini Index

Explanation Based Learning (EBL) -

- (i) Learn by explaining why an example is correct using prior knowledge
- (ii) Generalizes explanation to form a rule
- (iii) Produces compact, domain specific rules

Statistical Learning Methods -

- (i) Learn patterns using probabilistic & mathematical models
- (ii) eg:
 - Linear/Logistic Regression
 - Naive Bayes
 - SVM
 - Neural Networks
 - Probabilistic Graphical Models

Reinforcement Learning -

- (i) Agent learns by interacting with the environment
- (ii) Uses rewards & penalties to improve decisions
- (iii) Goal: maximize cumulative reward
- (iv) Key elements: state, action, policy, reward, value function
- (v) Methods: Q-Learning, SARSA, Deep RL (DQN, PPO)